Week 3, Day 6 (Weekend): Mini Project Kickoff

Prepared for Upendra Kumar Sharma | 5 to 6 hours | Goal: a running microservices skeleton with a gateway, JWT security, three business services, PostgreSQL, Kafka and Docker Compose, committed to GitHub

Today's Schedule

- **0:00 to 0:30** Read the project plan and fix your scope (sections 1 to 3).
- **0:30 to 1:00** Write the API and event contracts before any code (section 4).
- **1:00 to 1:40** Create the repository and infrastructure with Docker Compose (sections 5 and 6).
- **1:40 to 2:40** Build the auth service and gateway JWT validation (section 7).
- **2:40 to 4:10** Build the order and inventory service skeletons with database migrations (section 8).
- **4:10 to 4:50** Containerize the services and run everything with one command (section 9).
- **4:50 to 5:20** Smoke test through the gateway and write the README (sections 10 and 11).
- **5:20 to 5:50** Optional: revise this week's linked list and tree problems (section 13).

Today's rule: **a working, boring skeleton beats a half-built clever one.** Kafka events, Redis caching and Resilience4j are added in Week 4, so today you only prepare the ground for them.

1. What You Are Building

A small **order placement system**. A customer logs in, places an order, the system checks and reserves stock, and a notification is sent. It is deliberately simple in business logic, so all your effort goes into the engineering topics that interviewers ask about.

Why this project works for interviews

- It naturally needs every Week 3 topic: a gateway, inter-service communication, a saga across services, an outbox, idempotent consumers, resilience and caching.
- It is easy to explain in two minutes, and easy to draw on a whiteboard.
- It runs with one command, so a reviewer can try it in minutes. That matters more than size.

2. Architecture and Responsibilities

Services

- **api-gateway** (port 8080): the single entry point. Routes requests and **validates the JWT**. Built with Spring Cloud Gateway.
- **auth-service** (port 8081): issues JWTs. Reuse the secured API you built in Week 2, Day 6 (login, roles, BCrypt).
- **order-service** (port 8082): creates and reads orders. Owns the orders database, the outbox table and the order status lifecycle.
- **inventory-service** (port 8083): owns stock levels and reservations.
- **notification-service** (port 8084): consumes events and sends notifications (a log line or an email to a local mail catcher is enough).

Infrastructure

- **PostgreSQL:** one container for development, with a **separate database and user per service**. The rule is that no service reads another service's database, even if they share a server.
- **Kafka** (single node, KRaft mode): carries events between services from Week 4.
- **Redis:** added in Week 4 for caching. You can include the container now.

```
Client
  |
  v
api-gateway (validates JWT, routes)
  |--> auth-service        (login, issues tokens)
  |--> order-service  ---> orders DB, outbox --> Kafka (orders.created)
  |--> inventory-service <--- Kafka (orders.created) ---> inventory DB
  '--> notification-service <--- Kafka (order and stock events)
```

Design decision to be ready to explain: no Eureka or Config Server here

You built a discovery server and config server on Day 1. For this project, Docker Compose already provides **DNS-based discovery** (a container can reach order-service by its service name), and environment variables carry the configuration, which fits the twelve-factor approach and matches how Kubernetes works. Fewer moving parts means a faster start-up and easier debugging. If you want to show the Spring Cloud tools, keep them as an optional branch. In an interview, say which you chose and **why**, because that shows judgment.

3. Scope: What to Build When

Today (Week 3, Day 6)

- Repository, folder structure, Docker Compose with PostgreSQL, Kafka and Redis.
- Auth service issuing JWTs, gateway validating them, services also validating them.
- Order and inventory services with REST endpoints, Flyway migrations and the database tables (including the outbox table).
- Dockerfiles and one-command start-up. A smoke test and a README.

Week 4

- Days 1 and 2: Kafka events, outbox relay, idempotent consumers, dead letter topic.
- Days 3 and 4: Redis cache (cache-aside) and rate limiting.
- Day 5: Testcontainers integration tests, Resilience4j on the inventory call.
- Day 6: observability (Actuator, Micrometer), README polish, architecture diagram, push to GitHub.

Out of scope on purpose: a front end, payment integration, multi-tenancy. Keep the scope small so you finish.

4. Write the Contracts First

Define the interfaces before writing code. Put them in a docs folder in the repository, because they show design thinking.

REST endpoints (all through the gateway)

- **POST /auth/login:** body with username and password, returns an access token. Public.
- **POST /api/orders:** create an order with a list of items (sku and quantity). Role USER. Returns 201 with a Location header and the order in status PENDING. Accepts an **Idempotency-Key** header.
- **GET /api/orders/{id}:** read an order. A user can only read their own orders, and ADMIN can read any.
- **GET /api/orders:** list the caller's orders, paginated.
- **GET /api/inventory/{sku}:** read stock for a sku. Role USER or ADMIN.
- **PUT /api/inventory/{sku}:** set stock. Role ADMIN only.

Events (published from Week 4, defined now)

- **orders.created** (key: orderId): eventId, occurredAt, orderId, customerId, items.
- **inventory.reserved** and **inventory.rejected** (key: orderId): eventId, orderId, reason for rejection.

- **orders.confirmed** and **orders.cancelled** (key: orderId).
- Every event carries an **eventId** (for deduplication), a type, a version and a timestamp.

Order status lifecycle: PENDING, then CONFIRMED or CANCELLED. Today, orders stay PENDING. In Week 4 the saga moves them.

Error format: RFC 7807 Problem Details from every service, with a trace id.

5. Repository Layout and Git Habits

```
shop-platform/
  README.md
  .gitignore
  .env.example
  docs/
    api.md
    events.md
    architecture.md
  infra/
    docker-compose.yml
    postgres/init.sql
  services/
    api-gateway/
    auth-service/
    order-service/
    inventory-service/
    notification-service/
```

- A **monorepo** is easiest for a portfolio project. Each service is an independent Maven project with its own pom.xml and Dockerfile.
- **Never commit secrets.** Commit .env.example with placeholder values, and add .env to .gitignore.
- **Commit often with clear messages** (for example, feat: add order outbox table). A readable history is part of the portfolio.
- Use the same Java and Spring Boot 3 versions in all services.

6. Infrastructure with Docker Compose

Database initialization (infra/postgres/init.sql)

```
CREATE DATABASE auth;
CREATE DATABASE orders;
CREATE DATABASE inventory;
CREATE DATABASE notifications;
```

.env.example

```
POSTGRES_PASSWORD=change-me
JWT_SECRET=replace-with-a-random-value-of-at-least-32-characters
```

docker-compose.yml (infrastructure part)

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      timeout: 3s
      retries: 10

  kafka:
    image: apache/kafka:3.8.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: INTERNAL://:19092,EXTERNAL://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS:
INTERNAL://kafka:19092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
```

```
  redis:
    image: redis:7

volumes:
  pgdata:
```

Why two Kafka listeners: containers must reach Kafka at kafka:19092, while tools on your laptop use localhost:9092. A single advertised address cannot serve both. Check the image documentation for your chosen version, since the environment variable names can change.

Start the infrastructure with docker compose up -d postgres kafka redis, and confirm the three containers are healthy before moving on.

7. Auth Service and JWT at the Gateway

Auth service: copy the secured API from Week 2, Day 6 into services/auth-service. It needs POST /auth/login, BCrypt-hashed users with a roles claim, and a JWT that includes sub, roles, iat, exp (15 minutes) and iss.

Algorithm choice for today: use **HS256 with a shared secret** (JWT_SECRET) to get running quickly. The secret must be **at least 32 bytes**. State clearly in your README that production would use **RS256 with a JWKS endpoint**, so only the auth service holds the private key. Upgrading is a good stretch task in Week 6.

Gateway: validate the token before routing

```
@Configuration
@EnableWebFluxSecurity
public class GatewaySecurityConfig {

    @Bean
    SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http)
{
        return http
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .authorizeExchange(ex -> ex
                .pathMatchers("/auth/**", "/actuator/health").permitAll()
                .anyExchange().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
            .build();
    }

    @Bean
```

```
    ReactiveJwtDecoder jwtDecoder(@Value("${jwt.secret}") String secret) {
        SecretKey key = new
SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
        return NimbusReactiveJwtDecoder.withSecretKey(key).build();
    }
}
```

Routes (service names are Compose DNS names)

```
spring:
  cloud:
    gateway:
      routes:
        - id: auth
          uri: http://auth-service:8081
          predicates:
            - Path=/auth/**
        - id: orders
          uri: http://order-service:8082
          predicates:
            - Path=/api/orders/**
        - id: inventory
          uri: http://inventory-service:8083
          predicates:
            - Path=/api/inventory/**

jwt:
  secret: ${JWT_SECRET}
```

The property prefix for gateway routes changed in newer Spring Cloud releases, so confirm it in the documentation for your version.

Defense in depth: the services validate the token too

- The gateway forwards the Authorization header. **Each business service is also an OAuth2 resource server** with the same decoder (the servlet version, NimbusJwtDecoder.withSecretKey), so a request that bypasses the gateway on the internal network is still rejected.
- Map the roles claim to Spring authorities so that hasRole and @PreAuthorize work.

```
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter authorities = new
JwtGrantedAuthoritiesConverter();
    authorities.setAuthoritiesClaimName("roles");
    authorities.setAuthorityPrefix("ROLE_");
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authorities);
    return converter;
}
```

- **Data-level authorization:** in order-service, read the user id from the token subject and make sure users can only read their own orders.

8. Order and Inventory Service Skeletons

Common setup for each business service

- Dependencies: spring-boot-starter-web, validation, data-jpa, the PostgreSQL driver, **Flyway** (flyway-core and the PostgreSQL module), oauth2-resource-server, actuator.
- **Flyway migrations** in src/main/resources/db/migration create the schema. Set spring.jpa.hibernate.ddl-auto to validate, so Hibernate checks the schema but does not change it.
- **Global exception handler** returning Problem Details, and DTO records with validation (Week 2, Day 6).
- Actuator health endpoints, and server.shutdown=graceful.

order-service tables (V1__init.sql)

```
CREATE TABLE orders (
    id          uuid PRIMARY KEY,
    customer_id  text        NOT NULL,
    status       text        NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    version      bigint      NOT NULL DEFAULT 0
);

CREATE TABLE order_item (
    id        bigserial PRIMARY KEY,
    order_id  uuid NOT NULL REFERENCES orders(id),
    sku       text NOT NULL,
    quantity  int  NOT NULL CHECK (quantity > 0)
);
CREATE INDEX idx_order_item_order ON order_item (order_id);
```

```
CREATE INDEX idx_orders_customer_created ON orders (customer_id, created_at
DESC);

CREATE TABLE outbox (
    id             uuid PRIMARY KEY,
    aggregate_type text        NOT NULL,
    aggregate_id   text        NOT NULL,
    event_type     text        NOT NULL,
    payload        jsonb       NOT NULL,
    created_at     timestamptz NOT NULL DEFAULT now(),
    published_at   timestamptz
);

CREATE TABLE idempotency_key (
    key         text PRIMARY KEY,
    order_id    uuid        NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now()
);
```

inventory-service tables (V1__init.sql)

```
CREATE TABLE stock (
    sku        text PRIMARY KEY,
    available  int    NOT NULL CHECK (available >= 0),
    version    bigint NOT NULL DEFAULT 0
);

CREATE TABLE reservation (
    order_id   uuid        NOT NULL,
    sku        text        NOT NULL,
    quantity   int         NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (order_id, sku)
);

CREATE TABLE processed_message (
    message_id text PRIMARY KEY,
    handled_at timestamptz NOT NULL DEFAULT now()
);
```

What to implement today in order-service

- POST /api/orders: validate the DTO, check the Idempotency-Key (return the existing order if the key was seen), save the order as PENDING with its items, **write an OrderCreated row to the outbox in the same transaction**, and return 201 with a Location header.

- GET /api/orders/{id} and GET /api/orders with pagination, restricted to the caller's own data unless the caller is an admin.
- No Kafka yet. The outbox rows simply accumulate, which is exactly what you will relay next week.

What to implement today in inventory-service

- GET /api/inventory/{sku} and PUT /api/inventory/{sku} (admin only).
- Use an **atomic conditional update** for later reservations, for example UPDATE stock SET available = available - :qty WHERE sku = :sku AND available >= :qty, and check the updated row count.
- Seed a few skus with a migration or a startup script for testing.

notification-service: create an empty Spring Boot project with the health endpoint and a placeholder. It does its real work from Week 4.

9. Dockerfiles and One-Command Start-Up

Multi-stage Dockerfile (same pattern for every service; change the port)

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd -r -u 1001 appuser
COPY --from=build /app/target/*.jar app.jar
USER appuser
EXPOSE 8082
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

- **Why multi-stage:** the final image holds only the JRE and the jar, not Maven or source code, so it is smaller and has a smaller attack surface.
- **Copy pom.xml first** and resolve dependencies before copying the source, so Docker caches the dependency layer and rebuilds are fast.
- **Run as a non-root user,** and set the memory percentage so the JVM respects container limits.
- Use Java 21 only if your Spring Boot version supports it (Spring Boot 3.2 and later does). Otherwise use Java 17.

Adding the application services to docker-compose.yml

```
  api-gateway:
    build: ../services/api-gateway
    ports:
      - "8080:8080"
    environment:
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      - auth-service
      - order-service
      - inventory-service

  auth-service:
    build: ../services/auth-service
    environment:
      JWT_SECRET: ${JWT_SECRET}
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/auth
      SPRING_DATASOURCE_USERNAME: app
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
    depends_on:
      postgres:
        condition: service_healthy

  order-service:
    build: ../services/order-service
    environment:
      JWT_SECRET: ${JWT_SECRET}
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/orders
      SPRING_DATASOURCE_USERNAME: app
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    depends_on:
      postgres:
        condition: service_healthy
```

Inventory and notification services follow the same pattern with their own database names. Only the **gateway publishes a port to your machine**. The other services are reachable only inside the Compose network, which is itself a security decision worth mentioning.

Run everything: docker compose --env-file ../.env up --build from the infra folder, or adjust the paths to match your layout.

10. Smoke Test Through the Gateway

Run these checks and write the results in your README.

```
# 1. Log in and capture the token
curl -s -X POST localhost:8080/auth/login -H 'Content-Type: application/json'
-d '{"username":"alice","password":"password"}'

# 2. Call without a token: expect 401
curl -i localhost:8080/api/orders

# 3. Create an order with the token: expect 201 and a Location header
curl -i -X POST localhost:8080/api/orders -H 'Authorization: Bearer TOKEN' -H
'Content-Type: application/json' -H 'Idempotency-Key: demo-1' -d '{"items":
[{"sku":"SKU-1","quantity":2}]}'

# 4. Repeat step 3 with the same key: expect the same order, not a new one

# 5. Call an admin endpoint as a normal user: expect 403

# 6. Send a bad body (quantity 0): expect 400 with Problem Details
```

Also check that the order row **and** the outbox row exist in the orders database after step 3, and that a failure after the insert leaves neither (a quick test with a thrown exception).

11. README Template

A reviewer reads the README first and often reads nothing else. Include:

1. **Overview:** two sentences on what the system does and why it exists.
2. **Architecture diagram:** the text diagram above, or a Mermaid or image diagram.
3. **Tech stack:** Java, Spring Boot 3, Spring Cloud Gateway, PostgreSQL, Kafka, Redis, Docker.
4. **How to run:** copy .env.example to .env, then one docker compose command, and the URL of the gateway.
5. **API examples:** the curl commands from section 10.
6. **Design decisions and trade-offs:** database per service, gateway plus per-service JWT validation, HS256 now and RS256 later, Compose DNS instead of Eureka, outbox for reliable events, optimistic locking and idempotency keys.
7. **Roadmap:** what is built and what is next (Kafka saga, Redis cache, Resilience4j, tests).

12. Interview Talking Points from Today's Work

1. Why did you split the system into these services, and what does each one own?
2. Why does each service have its own database, and how do you prevent cross-database access?
3. Why validate the JWT both at the gateway and in each service?
4. Why HS256 for the prototype, and what changes with RS256 and JWKS?
5. Why did you skip Eureka and the config server here? When would you use them?
6. How do you keep secrets out of the repository and the images?
7. Why a multi-stage Docker build, and why a non-root user?
8. Why does the order and its outbox row share one transaction?
9. How does the Idempotency-Key make POST /orders safe to retry?
10. How would you evolve this from Compose to Kubernetes?

13. Optional DSA Revision (30 Minutes)

Redo from scratch, with a 10-minute limit each: Remove Nth Node From End, LRU Cache, Level Order Traversal, Validate BST. Then write the name of the pattern next to each (two pointers with a gap, hashmap plus doubly linked list, BFS with level size, range bounds).

14. End-of-Day Checklist

- [ ] The repository exists on GitHub with the layout above, and .env is ignored.
- [ ] Contracts for REST and events are written in the docs folder.
- [ ] docker compose starts PostgreSQL, Kafka, Redis and all services with one command.
- [ ] Login works, the gateway rejects requests without a token, and role checks return 403 where expected.
- [ ] POST /api/orders creates an order and an outbox row in one transaction, and the idempotency key works.
- [ ] Flyway migrations create the schemas, and Hibernate only validates them.
- [ ] The smoke test results and the README are committed.
- [ ] I can explain every design decision in section 12 in under a minute.

Tomorrow (Week 3, Day 7, weekend): system design introduction (the HLD framework) and a URL shortener, plus your Week 3 review and mock interview.